SECTION 05 · LOCAL DECISION MODELS

Ollama's Decision Endpoint and Nimble

Jev-style decisions on your own GPU: the /v1/systemone endpoint, how a decoder scores answer letters, and a personal-data check before a message leaves the company.

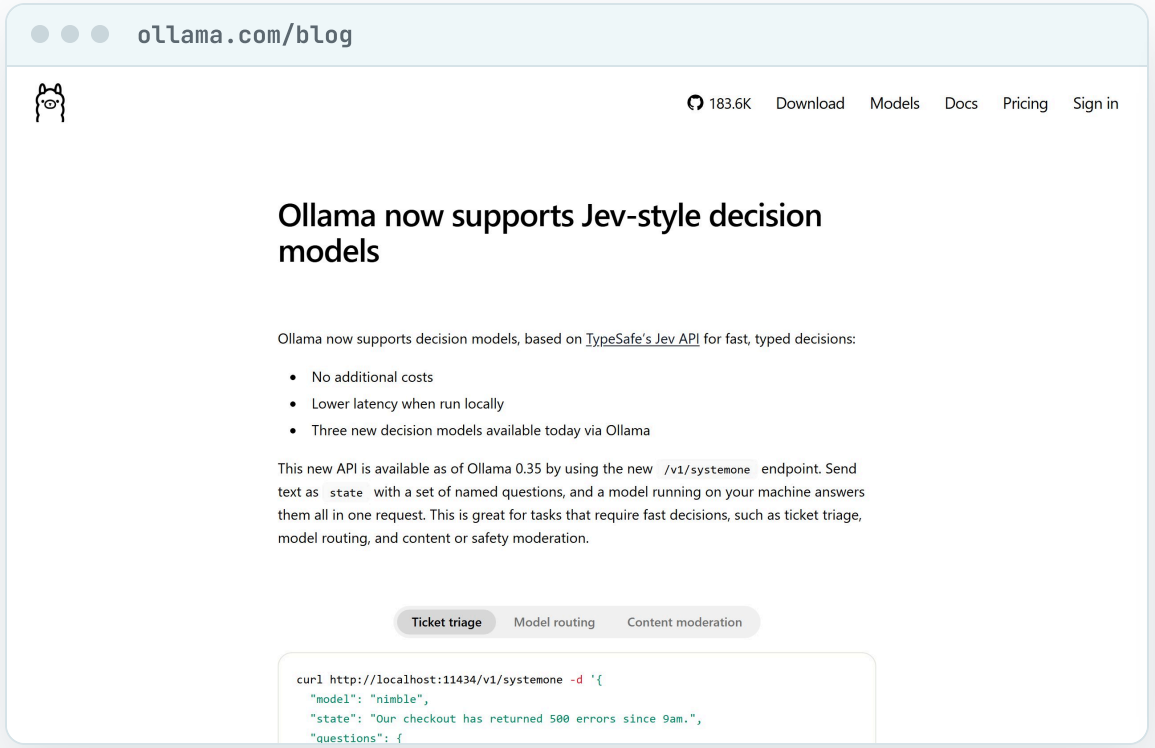
/v1/systemone | nimble | tev1 | Personal-data guard | Limits

KGP Talkie

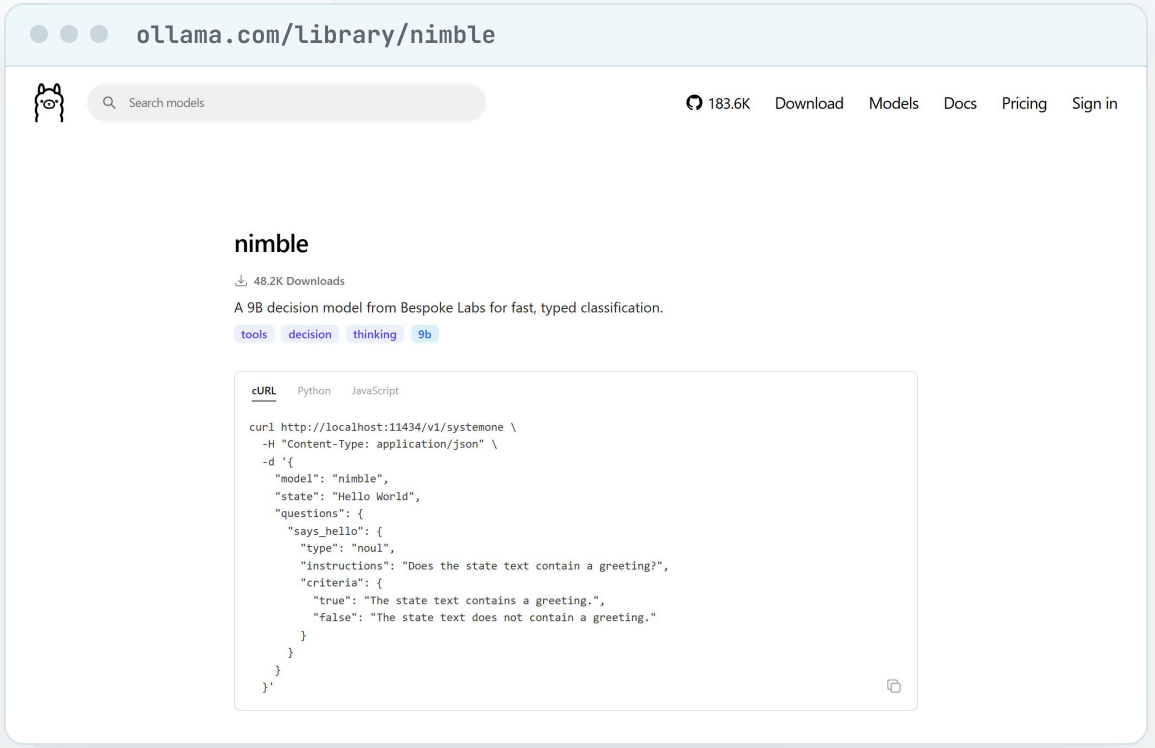
Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

Decision Models in Ollama



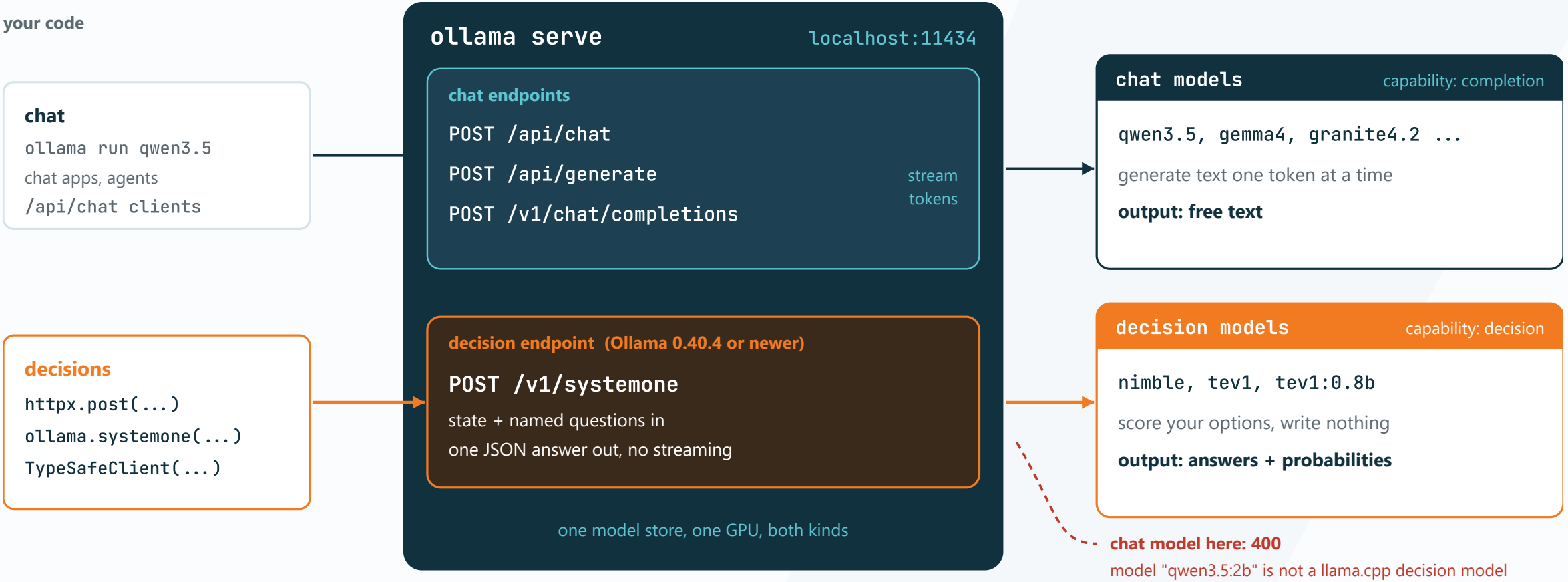
The Ollama blog post: the `/v1/systemone` endpoint



`ollama pull nimble`: a 9B decision model

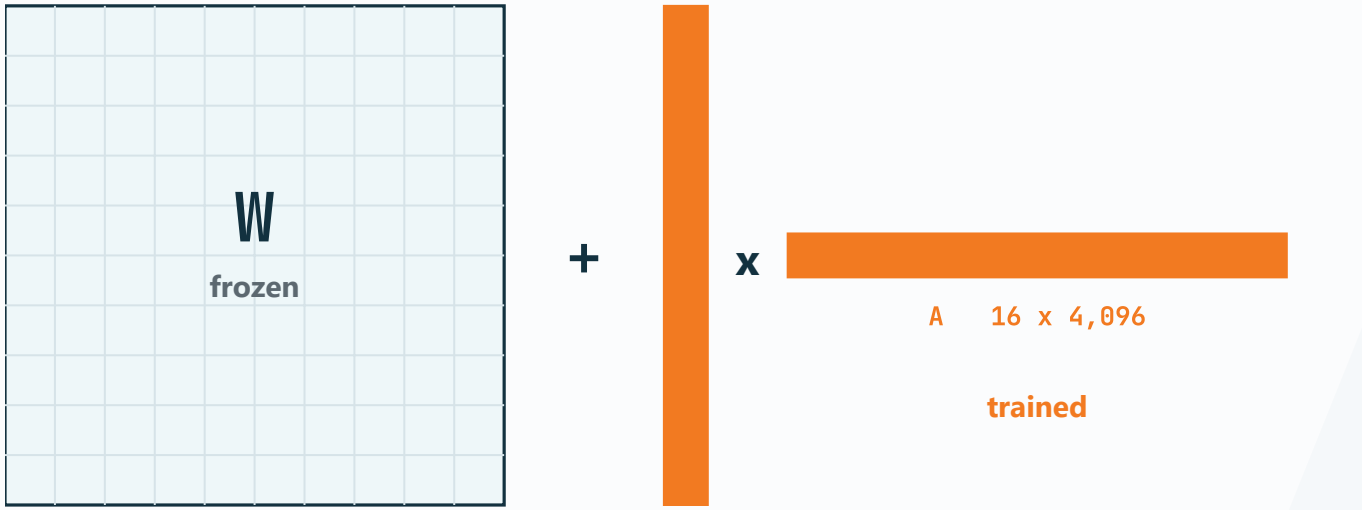
Three decision models in this section: nimble (Bespoke Labs), tev1 and tev1:0.8b (Together AI)

One Server, Two Kinds of Model



LoRA on a Frozen Base Model

one 4,096 x 4,096 weight matrix inside the base model



`W' = W + B A` rank 16: the update is squeezed through 16 numbers

after training, B A is added into W once: one ordinary model file

nimble's published recipe (Bespoke Labs)

base	Qwen3.5-9B, frozen
adapter	LoRA, rank 16
loss	cross-entropy over the allowed answer letters only
data	2,676 examples, 1 epoch; contrastive pairs: one fact changes, the label flips
ship	adapter merged, GGUF Q8_0
license	Apache 2.0

Model File Format and Size

Q8_0 stores weights in blocks of 32



32 weights x 1 byte (int8) = 32 bytes

fp16: 2 bytes

34 bytes per 32 weights = 8.5 bits per weight

each weight is rebuilt as int8 x scale when the model runs

nimble: 8,953,803,264 parameters

BF16 (2 bytes): 17,907,606,528 bytes	16.7 GB
Q8_0 (34/32): 9,513,415,968 bytes	8.9 GB
nimble's GGUF file: 9,527,503,648 bytes	8.9 GB

the extra: tokenizer, metadata, a few tensors kept at full precision

GB here = bytes / 1024^3. ollama list prints 9.5 GB because it counts decimal GB (bytes / 10^9). Same file, two units.

one .gguf file

header + metadata
architecture qwen35, 32 blocks, context, file type

tokenizer
vocabulary and merges, 248,320 tokens

tensors
every weight, Q8_0 blocks

+ Ollama adds a Modelfile layer:
system prompt, num_ctx, CAPABILITY decision, license

`ollama show nimble --modelfile`

Decision Models We Use on Ollama

nimble — Bespoke Labs

base	Qwen3.5-9B
parameters	**8.95B**
layers	32
	24 linear + 8 full attention
width	4,096
prompt fit	8,192 tokens
license	Apache 2.0

file size

8.9 GB

tev1 — Together AI

base	Qwen3.5-4B
parameters	**4.3B**
prompt fit	8,192 tokens
hosted	Tev1-4B-experimental
	on Together's serverless API
inspect	`ollama show tev1`

file size

4.3 GB

tev1:0.8b — Together AI

base	Qwen3.5-0.8B
parameters	**0.77B**
layers	24
	18 linear + 6 full attention
width	1,024
prompt fit	8,192 tokens

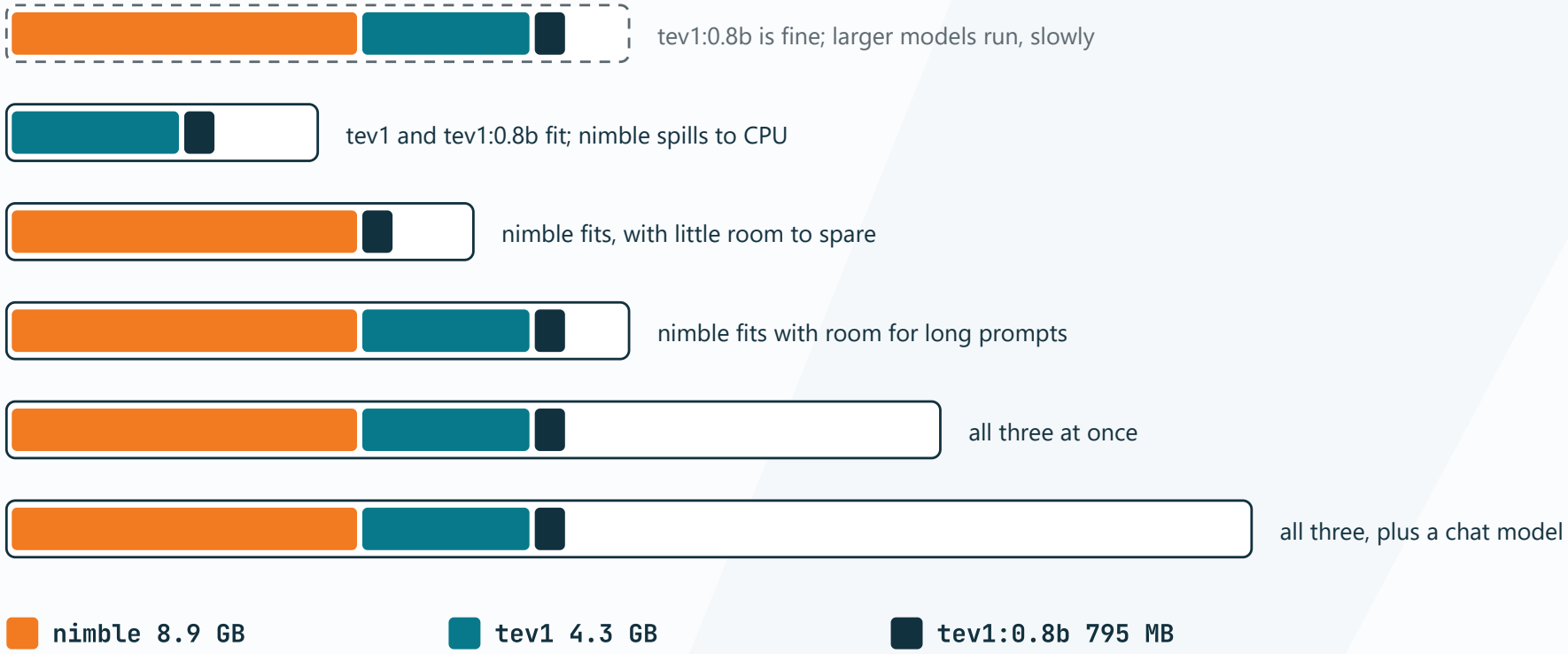
file size

795 MB

All three: decoder decision models, GGUF Q8_0, text only. File bars to scale. The decision filter on ollama.com lists more models.

Which Model Fits Your Hardware

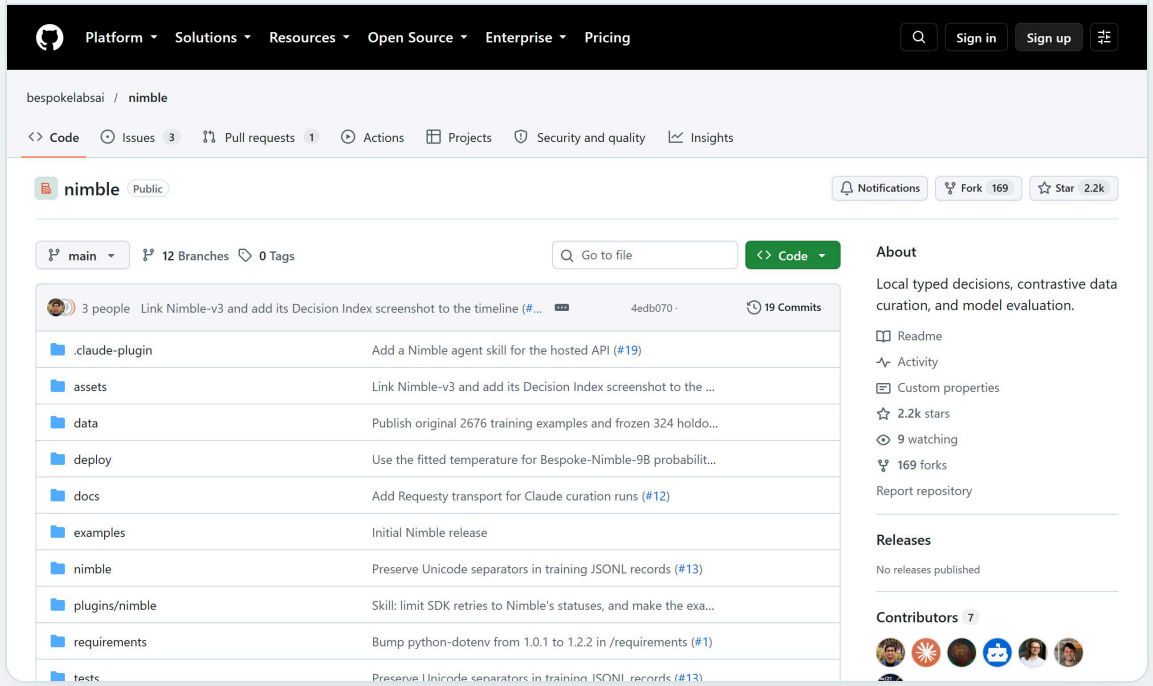
weights only, to scale; the prompt's working memory comes on top



If the weights do not fit, Ollama splits the layers between GPU and CPU: it still answers, more slowly. Apple Silicon shares one memory pool.

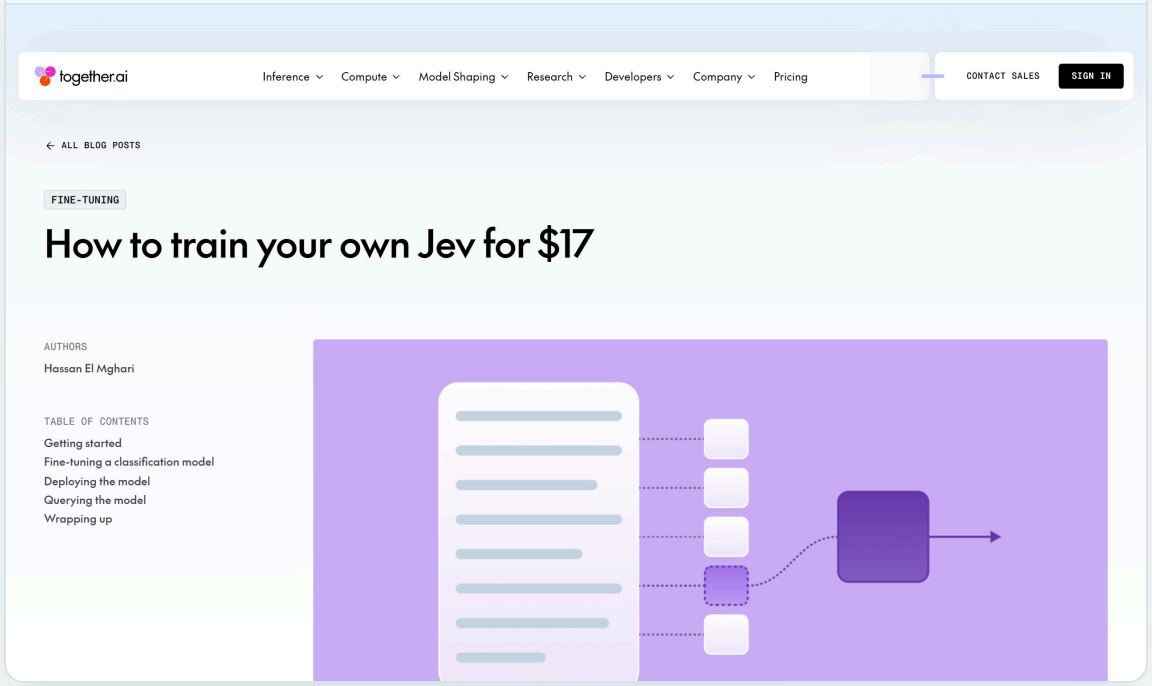
The Makers of nimble and tev1

github.com/bespokelabsai/nimble



Data, training recipe and evaluation. The nimble model is Apache 2.0

together.ai/blog



The tev1 recipe: fine-tune Qwen3.5 on about 38,000 examples

Both are Qwen3.5 fine-tunes; Bespoke says nimble learned from its own labels, not distilled from Jev

Parts of a Decision Request

request

```
POST http://localhost:11434/v1/systemone
{
  "model": "nimble",
  "state": "Hi Priya, your replacement card
           ending 4417 ships to 22 Elm St ...",
  "questions": {
    "personal": {
      "type": "noul",
      "instructions": "The message contains
        personal data about a specific person."
    }
  }
}
```

question names are yours: they come back as the keys of answers

response

```
{
  "model": "nimble",
  "answers": {
    "personal": {
      "type": "noul",
      "noul": P(true), 0 to 1
    }
  },
  "usage": {"input_tokens": n, "output_tokens": n}
}
```

The same JSON as hosted Jev's /v1/systemone:
only the host and the model name change

Noul, Choice and Score Answers

noul — yes or no

you send

```
"instructions": a statement
"criteria": {"true": ..,
             "false": ..}  optional
```

you get back

`noul`	P(true)

one number:



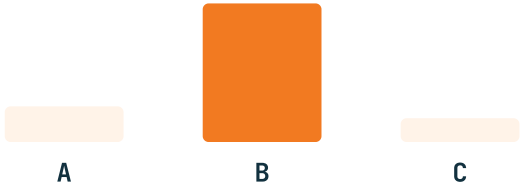
choice — 2 to 255 options

you send

```
"criteria": {
  "clean": "..",
  "personal_data": "..",
  "internal_info": ".."}
```

you get back

`choice`	the top option
`probabilities`	one per option
`confidence`	0 to 1



score — 2 to 10 levels, low to high

you send

```
"criteria": [
  "none",
  "some",
  "serious"]
```

you get back

`score`	expected level
`legend`	level to text
`probabilities`	one per level
`confidence`	0 to 1



How a Decoder Decision Model Answers

the prompt for one question (category), as tokens (· is a space), layout simplified

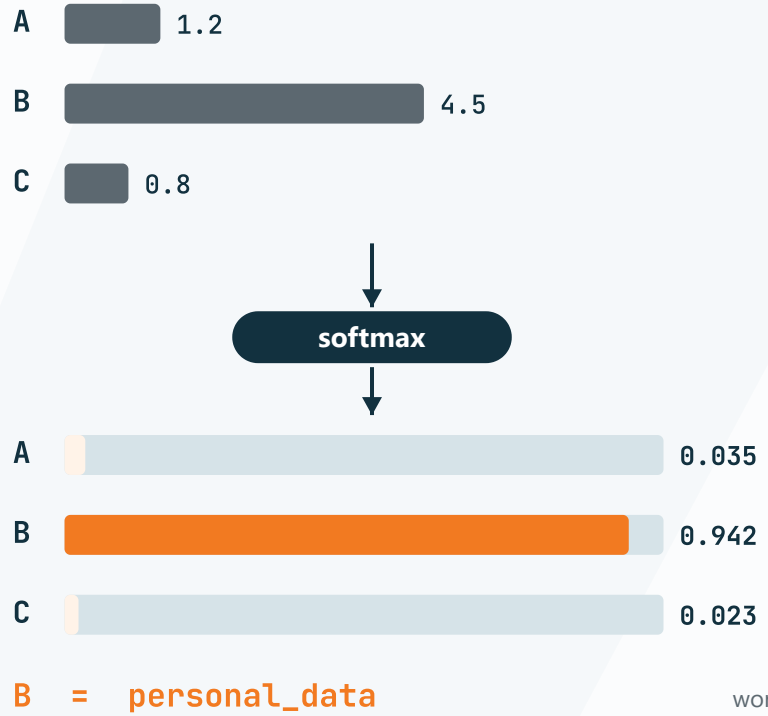
system	Class ify the context using the supplied schema ...
schema	category : ... A = clean B = personal _data C = internal _info
context	Hi Pri ya , your card ending · 4 4 1 7 ships to ...
field	category

? the next token: one letter

nimble: Qwen3.5-9B with a LoRA, merged
32 layers, reads the prompt left to right
hidden state at the last position: 4,096 numbers

output head: 248,320 vocabulary rows
only the rows for A, B, C are read

logits for the answer letters



worked example logits

No text is generated: the model's next-token scores for the allowed letters **are** the answer.

Letter Logits to Probabilities

choice: softmax over the letters

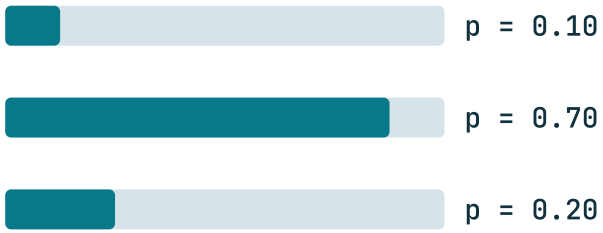
code	option	logit z	e^z	p = e^z / sum
A	clean	1.2	3.32	0.035
B	personal_data	4.5	90.02	0.942
C	internal_info	0.8	2.23	0.023
			sum 95.56	1.000

choice = the option with the largest p: personal_data

the full vocabulary never competes: only the letters you allowed are normalised

Dividing z by a temperature T > 1 flattens p, same pick

score: expected level




```
score = sum of i x p_i
      = 0 x 0.10 + 1 x 0.70 + 2 x 0.20
      = 1.10
```

fractional: a little above "some"

legend maps each level back to its text

worked example probabilities

Ollama's Confidence, Worked Out

choice: above an even guess

clean		0.0052
personal_data		0.9786
internal_info		0.0162

n = 3 options, an even guess is 1/n = 0.3333

```
confidence = (p_max - 1/n) / (1 - 1/n)
= (0.9786 - 0.3333) / (1 - 0.3333)
= 0.6453 / 0.6667
```

= 0.9679 the returned confidence

score: close to the peak

0 none		0.0126	1 level away
1 some		0.8369	peak
2 serious		0.1505	1 level away

spread: each probability times its distance from the peak

```
confidence = max(0, 1 - spread / even_spread)
spread = 0.0126 x 1 + 0.1505 x 1 = 0.1631
even_spread = (1 + 0 + 1) / 3 = 0.6667
= 1 - 0.1631 / 0.6667
```

= 0.7553 the returned confidence

The same formulas as hosted Jev: Ollama returns the same two values.

0 is an even guess (choice) or an even spread (score), 1 is certain.

THE SAME DISTRIBUTION CAN MEAN VERY DIFFERENT THINGS

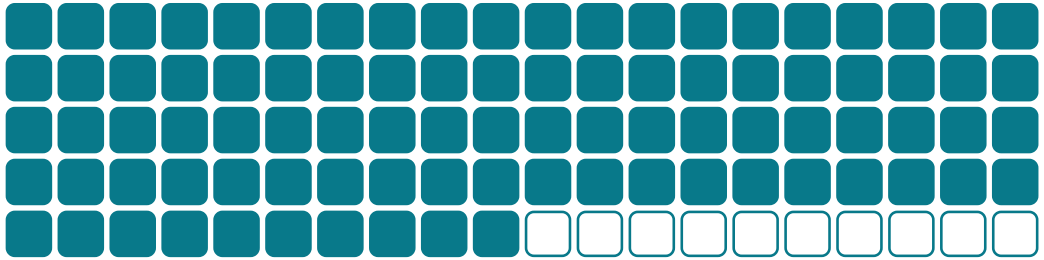
High Confidence Is Not Accuracy

model X

same answer distribution

A	0.90
B	0.07
C	0.03

100 answers that all looked like this. How many were right?



90 of 100 right

calibrated: 0.90 means right 9 times in 10

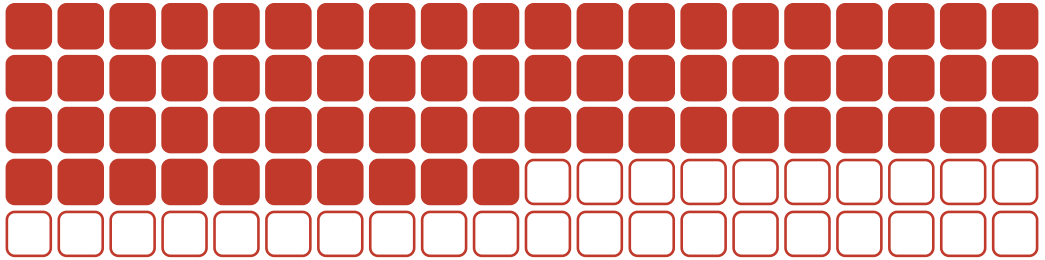
confidence alone cannot tell X from Y

model Y

same answer distribution

A	0.90
B	0.07
C	0.03

100 answers that all looked like this. How many were right?



70 of 100 right

over-confident: 0.90, right 7 times in 10

confidence alone cannot tell X from Y

A check Bespoke published for nimble: average top probability **0.89**, answers right **73%** of the time.

Same Code, Three Backends

your code: identical apart from the constructor

```
from typesafe_sdk import TypeSafeClient, Noul

nimble = TypeSafeClient(
    base_url="http://localhost:11434",
    api_key="local",    # ignored, but required
    model="nimble")

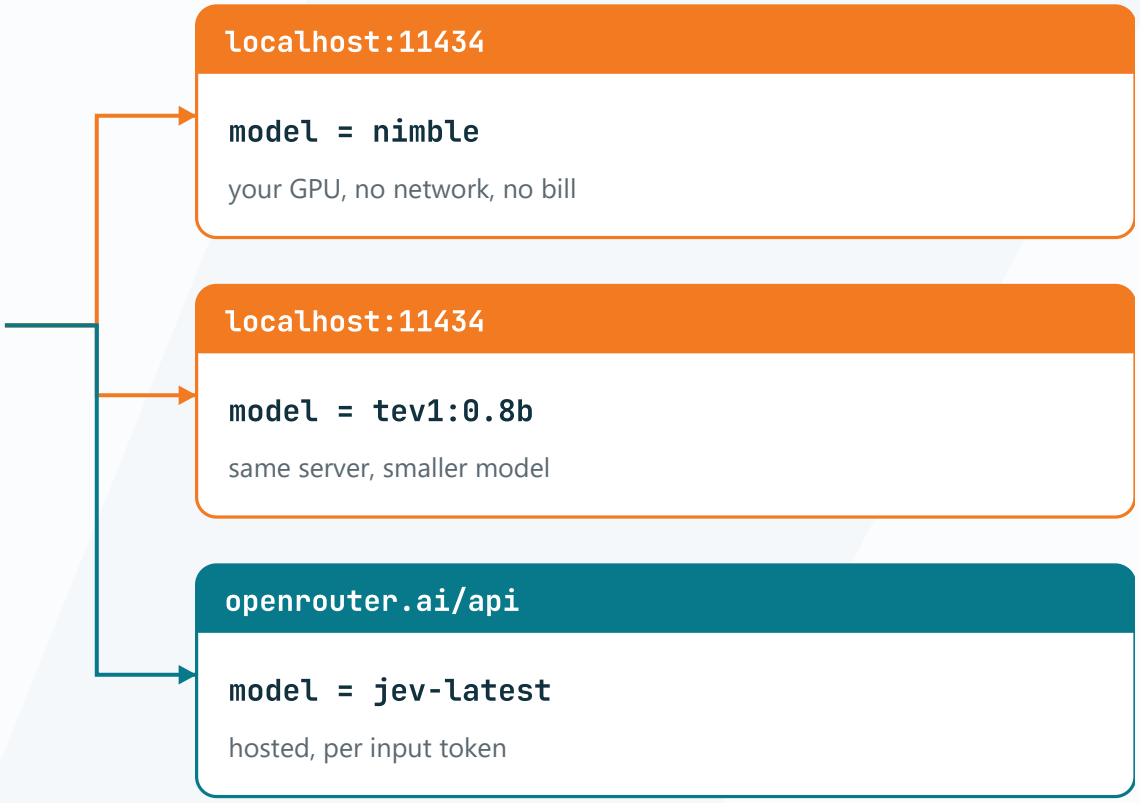
jev = TypeSafeClient()    # .env: OpenRouter

r = nimble.system_one(text, {"personal": PERSONAL})
```

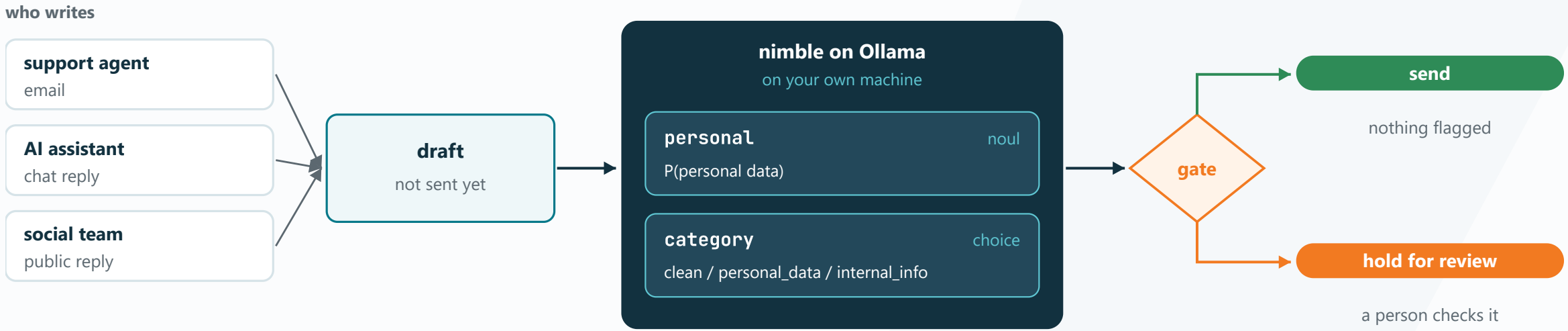
SystemOneResponse, the same typed object from every backend:

`r.model, r.answers[...].noul, .choice, .probabilities, r.usage`

the SDK posts to {base_url}/v1/systemone



Use Case: Checking Outbound Messages



who uses this

ops / support lead	fewer leaks, review only what is flagged
compliance	personal data never leaves without a check
developer	one local call before send(), no data to a third party
analyst	which channels and teams get held most

Writing the Personal-Data Question


```
"personal": {"type": "noul",  "instructions": "The message contains personal data about a specific person.",
  "criteria": {"true": "a phone number, home address, card digits, ID or passport number, personal email, DOB, health"
               "false": "no personal data; a first name, an order number or our own helpline does not count"}}
```

true: hold

"... we tried calling you at 555-0142 ..."

"... the refund went back to the visa ending 4417"

"... thanks for telling us about your epilepsy diagnosis"

"... 9 Sample Towers, Andheri West, Mumbai"

false: fine to send

"Hi Ankit, I've escalated your ticket ..."

"Your order #A-20931 is out for delivery"

"... call our helpline on 1-800-555-0100"

"you can update your shipping address any time"

The false side names the near misses: they are where an undescribed question goes wrong.

Send or Hold for Review

hold if either signal fires

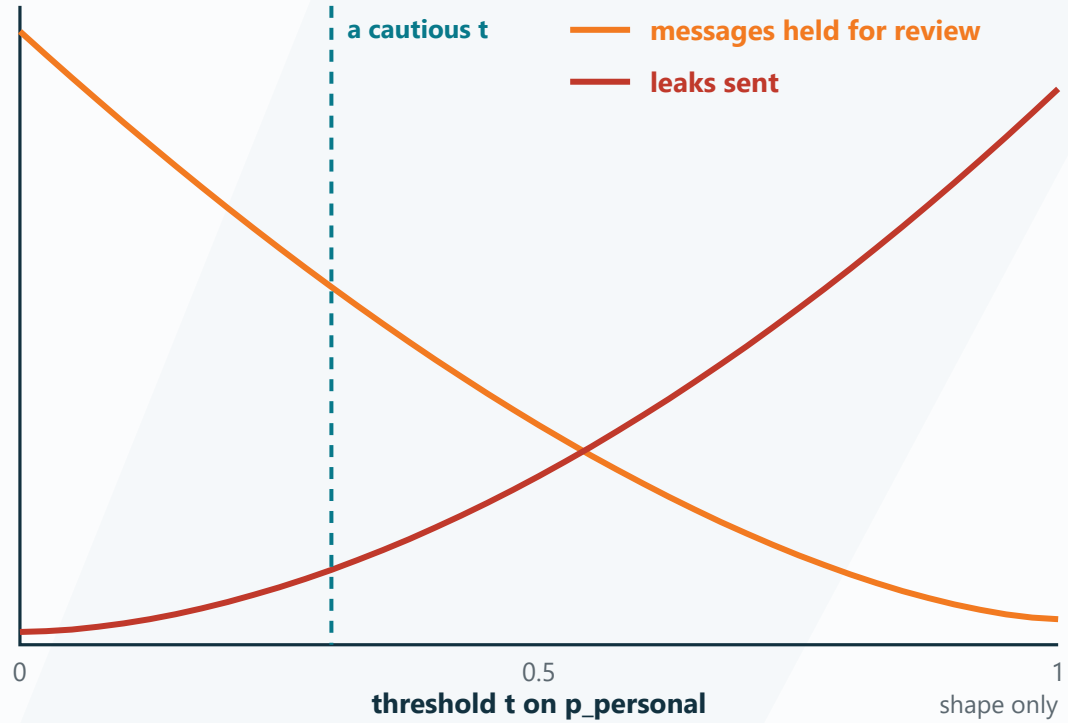
```
hold = (p_personal > t)
     | (category ≠ "clean")
```

two questions, two chances to catch it:

the noul catches personal data the choice calls clean, the choice catches internal information the noul is not asked about

a leak and a review cost different amounts:

one leaked card number vs one minute of a reviewer's time. Set t below 0.5 when leaks cost more.



The Limits of /v1/systemone

1 to 64

questions per request

400

```
questions must contain 1 to 64 fields
```

8,192

tokens per prompt, nimble and tev1

400

```
request (8995 tokens) exceeds the
available context size (8448 tokens),
try increasing it
```

nimble's 8,194 is rounded up to 8,448

64 KB

text and schema

413

```
text and schema must not exceed 64 KB
```

images do not count

text only

no images for these models

400

```
registry.ollama.ai/library/nimble:latest
does not support vision
```

2 to 255

options per choice question

400

```
too many options (256), this model
supports at most 255
```

tev1 was trained on 2 to 24: stay in that range

2 to 10

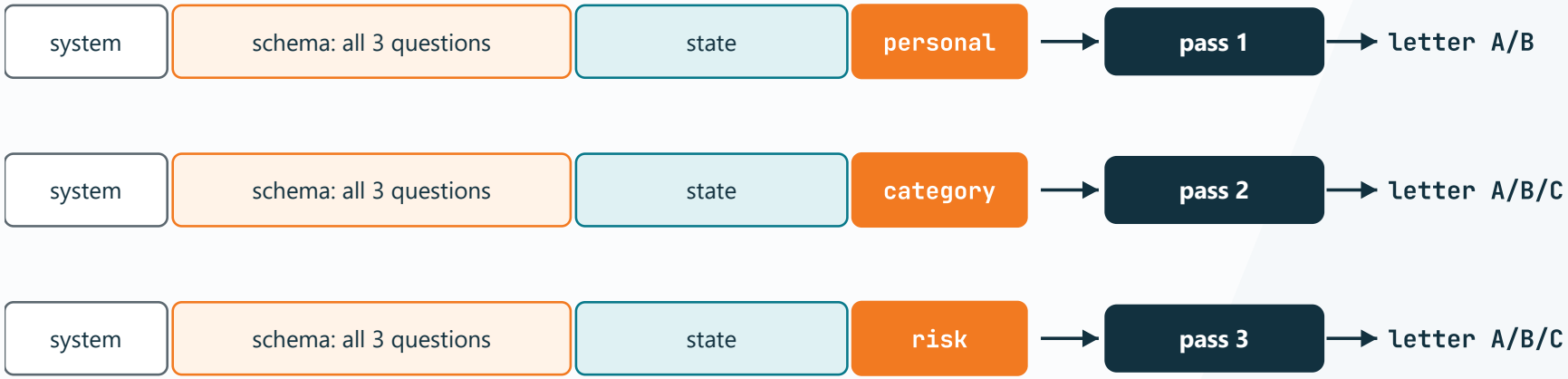
levels per score question

400

```
score criteria must be an array of 2 to
10 levels
```

One Prompt per Question

one request, three questions: Ollama builds three prompts



every prompt repeats the system text, the full schema and the state

Shared prefix (Bespoke's own MLX scorer)

read system + schema + state once, keep the cache,
then branch a short suffix per question

Ollama's endpoint: the full prompt per question

Encoder decision models

one row per question, all rows in one forward pass, no next token to predict

the encoder route from an earlier section

Why 64 Questions Cost More Locally

what every prompt holds

the system text

the state

all n questions

the field selector

---

one prompt, n questions:

`fixed part + n questions`

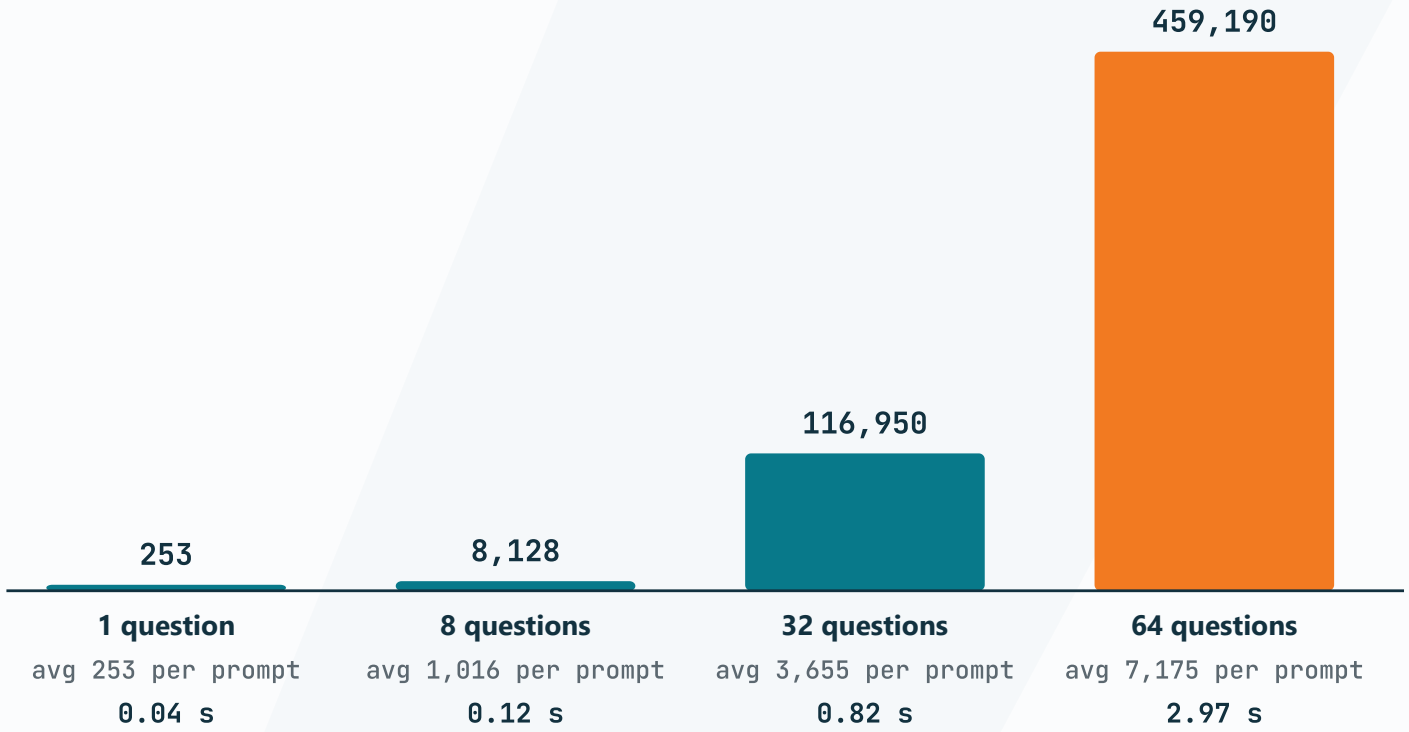
whole request, n prompts:

`n x (fixed + n questions)`

grows with n squared: 32 to 64 questions
is 3.93 times the input tokens

one prompt must still fit the model:
8,192 tokens for nimble and for tev1

measured on nimble: input tokens for the whole request



Long Inputs Are Rejected, Not Cut

silent truncation (some runtimes)



the end is cut off: the field selector and the last messages vanish,
and an answer still comes back, looking normal

Ollama /v1/systemone



nothing is cut: the whole request is rejected with the token count,
so you find out at once and decide what to send

Fixes: judge only the newest message, split the thread, or use a model with a larger prompt fit.

Comparing Backends Fairly

same 200 messages, same Choice, one message at a time

nimble on Ollama
local GPU, the full prompt per question



tev1:0.8b on Ollama
local GPU, a smaller model



hosted Jev
OpenRouter, then TypeSafe, and back



time for one message: widths are a sketch, the notebook measures the real thing

measure both on your own data: accuracy per intent, and seconds per message end to end

network time does not shrink with a faster model; prompt reading grows with the questions you ask

Choosing Local or Hosted

if this matters most	lean to	because
data may not leave the machine	**local**	hosted sends the text to a provider
probabilities you will threshold	**hosted Jev**	trained for calibration (RLCD); local: re-check thresholds
no per-call bill, steady high volume	**local**	you pay for the GPU instead
no GPU, or a fleet of small machines	**hosted**	or tev1:0.8b on CPU
many questions per message	**hosted, or fewer questions**	locally every question re-reads the prompt
long inputs	**hosted Jev**	about 32k by default, vs 8,192 per nimble prompt

SECTION 05 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie